
Autonomous and Adaptive Systems Project

Lorenzo Zoccadelli

Computer Engineering, University of Bologna
lorenzo.zoccadelli@studio.unibo.it

Abstract

This project investigates how multiple autonomous agents can learn and act in Stag Hunt and Escalation social dilemmas. It explores the behaviour of two agents trained with reinforcement learning via DQN and under which conditions they can cooperate in both environments, changing the rewards magnitude and the size of the environment. Then some experiments were done using small LLMs (3B, 8B and 14B of parameters) acting as agents through prompt-based inference. As result, while severe punishments discourage cooperation and promote selfish and risk-averse behaviours in personal reward settings, shared global rewards move agents toward more cooperative strategies. Additionally, LLM experiments demonstrate that these models can also perform fairly well without finetuning.

1 Introduction

1.1 Social Dilemmas

Social dilemmas are an area of game theory that offers the possibility to understand how individuals act when they have to choose between cooperation or defection in different scenarios. They are also important for studying how independent autonomous agents interact and under which conditions they learn to cooperate or not. Stag Hunt is one of the most known examples: agents have to choose between cooperating to catch a stag (getting a high reward), with the risk of being defected against the other agent (getting a certain penalty), or collecting a plant alone (getting a low reward). Another one is the Escalation scenario: the two agents need to cooperate to catch a stag starting a streak, earning a reward for every step of the streak. If one agent defects while the other persists, the abandoned agent obtains a negative reward proportional to the length of the streak. The main goal of this project is to train a pair of autonomous agents to play Stag Hunt and Escalation environments, studying under which conditions they cooperate or converge to a more selfish or suboptimal behaviours. The implementation of the environments used was the one created by [1], which provides implementation for both Stag Hunt and Escalation scenarios (the harvest scenario was not considered in this project).

1.2 Reinforcement Learning Agents

Mostly in all the experiments of this project the agents were supposed to learn how to act into the environment via reinforcement learning. In such a way, provided only with a representation of the state and the rewards related to their actions, they have to learn themselves how the environment works and how to cooperate or defect with no prior knowledge.

1.3 LLM Agents

As the capabilities of LLMs improve, also of the smaller ones, they're increasingly used also as agents acting in environments like this, using them both after finetuning or directly just via prompt-based inference, designing ad hoc prompts leveraging on their intrinsic general knowledge and "reasoning capabilities". Therefore, it's interesting to experiment how these models can perform in these kind of

scenarios without any further training. For this reason, in this project some experiments using LLMs as agents were also done, performing no finetuning, just providing them prompts explaining how the environment works. This kind of approach was used only for the Stag Hunt environment.

2 Methodology

2.1 DQN

DQN was chosen as RL algorithm. It is an off-policy value approximation method in which the agent approximates the Q value of each state-action pair through a neural network. Agent acts ϵ -greedy over the Q values during training and purely greedy for evaluation.

Update The agent learns by improving the $\hat{q}_w$ network (and so also the policy based on it) parametrized with some weights w , aiming to minimize the difference between the predicted Q value and the target U_t (1), computed using the reward and the Q value of the best action in the next state. The weights are then updated via semi-gradient descent to minimize the MSE between U_t and $\hat{q}(S_t, A_t, w_t)$, minimizing the TD-error (2).

$$U_t = R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} \hat{q}(S_{t+1}, a, w_t) \quad (1)$$

$$w_{t+1} = w_t + \alpha(U_t - \hat{q}(S_t, A_t, w_t)) \nabla_w \hat{q}(S_t, A_t, w_t) \quad (2)$$

Experience Replay To avoid a too high correlation between the updates, DQN introduces also the idea of experience replay. After every time step the $(S_t, A_t, R_{t+1}, S_{t+1})$ tuple is stored in a buffer and not directly used to improve the network. Experiences are then randomly sampled from the buffer, usually in batch, reducing the correlation between updates.

DQN Improvements The vanilla DQN implementation was extended with some simple improvements well known in the literature:

- **Double DQN** [6]: A target DQN, a delayed copy of the main one, is used. It is kept frozen and is updated by copying the main network’s parameters every n updates. When computing the target value, the main DQN is used to select the best action, while the target DQN is used to evaluate it. This prevents the targets from shifting constantly, making the training more stable. Furthermore, decoupling action selection and evaluation reduces the overestimation bias typical of standard DQN.
- **Dueling DQN** [7]: The network is changed to make it predict separately the state value (how good a certain state is on average) and the advantages of each action with respect to the average. These values are then combined to compute the Q values, but this layout helps the network to learn the average value also if only one action at a time is used for the optimization and in the states in which actions have similar values.
- **Prioritized ER** [2]: The experience replay buffer is extended to add a priority (proportional to the last TD-error) to each experience. The priorities are used to sample experiences, so that the ones for which the model was more wrong are sampled more frequently.

2.2 Stag Hunt and Escalation

State Representation It was chosen to give agents the entire visibility of the grid, representing the state as an array of normalized relative positions. The Gymnasium Stag Hunt environment [1] actually provides the state as the absolute coordinates of the entities in the grid (the agents, the stag and, only in Stag Hunt, the plants). These coordinates are converted to relative positions of each element with respect to the agent: difference along the x and y axis, giving the agent the idea of how far every object was in the 4 directions. Then these values are normalized dividing by respectively the width and the height of the grid. This representation turned out to be the one that worked better (also raw coordinates and bitmap representations have been tried). In addition, this representation scales quite well when increasing the size of the grid.

Rewards For both scenarios the reward structure originally implemented by the environment was used, changing only the magnitudes of the rewards to study the behaviour of the agents under various reward conditions. Then also some tests with shared rewards were made, to further study if changing the objective of the agents to maximize a global shared reward (instead of their personal reward) changed their behaviour. In this case, each agent is provided with the sum of the rewards of both the agents.

Agents structure It was decided to keep agents completely separated and independent. In particular each agent has its own network and doesn't share weights nor experiences with the other. In this way the agents are free to learn to behave independently, potentially in different ways depending also on the other agent's actions. This approach is more coherent with a social dilemma scenario.

2.3 LLMs for Stag Hunt

In these tests, LLMs were used only for inference. Some models from the Mistral 3 Reasoning family were used, in particular the versions with 3B, 8B and 14B of parameters, all quantized to 8 bit GGUF [4] [5] [3]. These types of models are trained to enhance reasoning capabilities, performing better than other generic instruct models of similar dimension in a problem like Stag Hunt.

To make it able to interact with the environment, at every time step, a conversation-like prompt is provided to the model that has to complete it with its reasoning and the decision about the action to take. First the model is prompted with an explanation of the rules, the dynamics of the environment and the objective of the agent. Then it is provided with a description of the current state, after which is asked to think about the action to perform. To do so the "Let's think step by step" prompt is used to trigger a Chain of Thought (CoT), after which the model eventually decides which action to take, outputting a specific token ('L', 'R', 'U', 'D', 'I'). To ensure that the model respects the structure of the conversation, a handwritten few shot prompt is prepended to every interaction. This allowed it to write its decision directly in the response, after the CoT, and then retrieve it by parsing the response string. This method was more practical than splitting the conversation and using the logits of the last token (given the variable length of the generated reasoning). After the few shot prompt, the model is provided only with the current state, so it does not have the knowledge of previous history.

Regarding the state, the environment was described to the agent providing the relative position of the entities instead of their absolute positions or a grid-format representation. This approach minimised the need for too complex mathematical and geometric reasoning, giving the model a cleaner idea of its position with respect to all the elements of the grid.

3 Results

For all the experiments conducted on Stag Hunt and Escalation, a 5x5 grid was used and the episodes were set to last 200 time steps unless otherwise stated. All plots show the evaluation results obtained by evaluating the Q-network with the greedy policy after each training episode.

3.1 Stag Hunt with DQN

In the Stag Hunt environment, experiments were conducted changing the magnitude of the rewards in both a personal and global reward maximization scenario. Then, fixing the values of the rewards, it was tested how scaling the dimensions of the grid affected the training. In all the experiments the foraging reward was set to 1.0, stag reward to 5.0, the number of plants to 2 and the behaviour of the stag to follow the nearest agent and not to "run away" after mauling.

Reward Conditions Some experiments were conducted setting different values for the mauling punishment (mp). As expected, in the independent reward maximization configuration (Figure 1), increasing the magnitude of the punishment makes it more difficult for the agents to learn to cooperate. In particular, while at small mp values agents learn quite early to cooperate, almost never foraging, with $mp = -3.5$ they first start foraging independently and, only after half of the training, learn to cooperate instead. Conversely, with the most severe $mp = -5.0$ agents do not cooperate at all, ending up in a quite stable foraging behaviour. Results change when the shared reward configuration is used (Figure 2). In this case the agents eventually learn to cooperate perfectly in all settings.

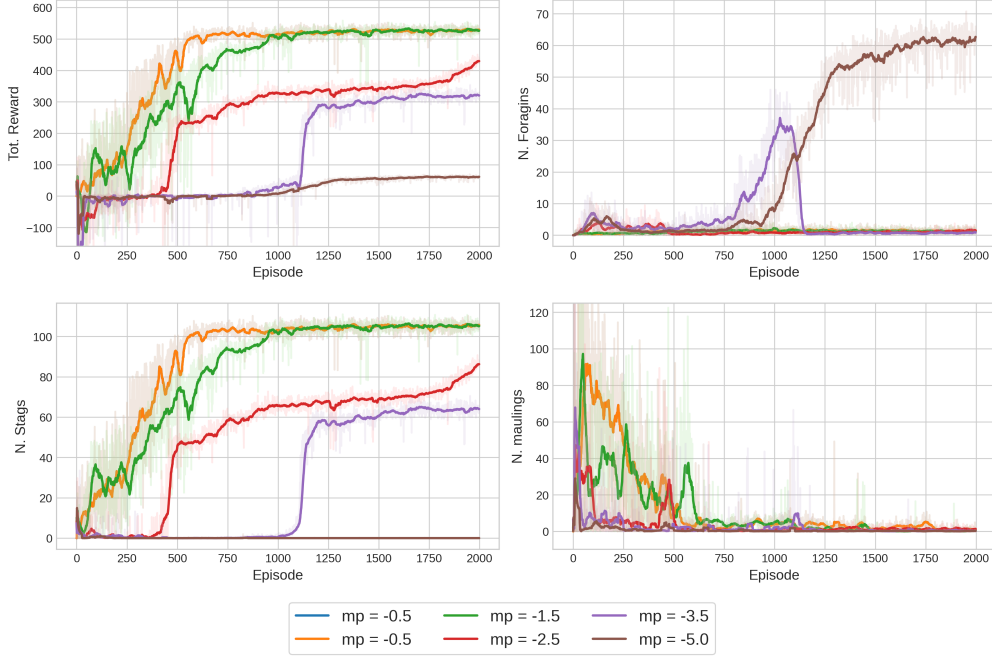


Figure 1: Episode total rewards, stags, foragings and maulings of one of the agents in the default personal reward configuration, for different mauing punishment values.

Size Scaling Fixing $mp = -1.5$ (that led to cooperative behaviour), it was studied how this approach scaled with different grid sizes (Figure 3). In this case, agents are able to learn cooperative behaviour in all the configurations. The number of captured stags and the episodic rewards decrease as the size increases because it takes more time for the agents to reach the stag, making rewards more sparse. In these tests a pair of agents was trained for each grid size. No generalization training was performed to make a single agent able to play with various sizes.

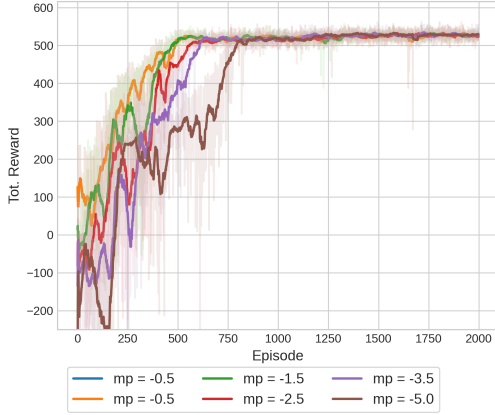


Figure 2: Episode total global rewards changing mp. The values are divided by 2 to make them comparable with the other results.

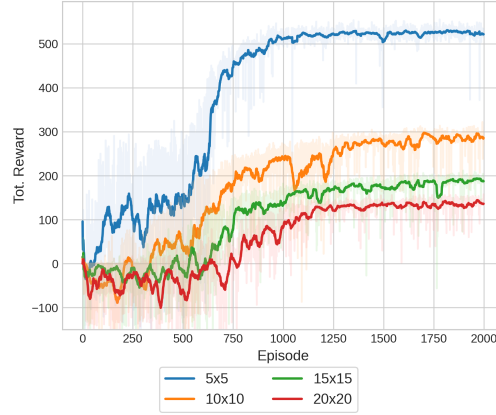


Figure 3: Episode total rewards achieved by one of the agents on different grid sizes.

Looking at agents playing, it's worth pointing out how, in general, they learn to proactively step together toward the stag, reducing the amount of time needed to catch it. In fact, during some trainings (mainly testing also PPO) agents didn't explore enough to find this solution and ended up in a lazier one, staying still together in a corner waiting for the stag to reach them, which is actually a sub-optimal strategy. Though, it must also be said that in the $mp = -3.5$ case the agents have a strange behaviour, preferring to move along the border of the grid. Anyway, it may be that a longer training and more exploration could improve this behaviour.

3.2 Escalation with DQN

Regarding the Escalation environment, only experiments under different reward conditions were performed. In this case, the only parameter to tune was the streak break punishment multiplier (bp). Also for this environment tests were conducted with personal (Figure 4) and shared (Figure 5) reward maximization configurations to see the difference. The results are similar to the Stag Hunt ones: the heavier the punishment is, the more the agents struggle to learn a cooperative behaviour and, also in this case, making them share the rewards allowed them to cooperate better in all the configurations.

Observing agents playing, with $bp = 1.0$ (personal reward maximization) agents cooperate for few consecutive steps and sometimes defects together (with no unilateral defections), apparently learning what the other will do in a certain situation. With $bp = 2.0$ instead, they learn a good "diffident" strategy: they cooperate to catch the stag when it is sleeping, that is a riskless cooperation (based on how the environment is implemented), and they both defect the following time step, so that no one would take the penalty, repeating this strategy until the end of the episode.

It can be seen that with shared rewards (in particular for $bp = 2.0$, for which it also happens in the default case), the learning seems a bit unstable: agents learn to cooperate quite often but there are some episodes in which they seems to "forget" the cooperative strategy and don't cooperate at all, recovering the optimal behaviour the next few episodes. Actually, for $bp = 2.0$ the training is very unstable, while this happens in few episodes in the other cases.

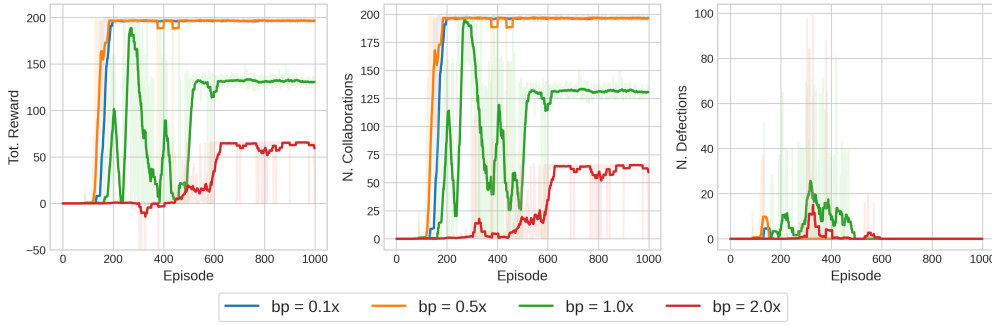


Figure 4: Results obtained in the personal reward maximization configuration on the Escalation environment changing the streak break multiplier. Total episodic rewards, number of collaborations and defections suffered by one of the agents.

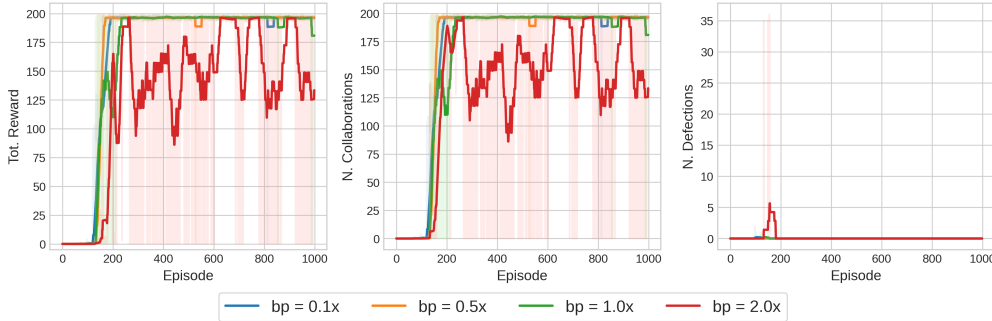


Figure 5: Results obtained in the global reward maximization configuration on the Escalation environment changing the streak break multiplier. Global episodic rewards, number of collaborations and defections suffered by one of the agents. The episodic reward is divided by 2 to make it comparable with the other experiments.

3.3 Stag Hunt with LLMs

LLMs of different sizes were tested playing the Stag Hunt environment both against themselves (i.e. another LLM agent using the very same model) and against a pretrained DQN agent (one of the previous experiments). In both cases the LLM achieved fairly good total episodic rewards, succeeding in cooperating with the other agent, with bigger models achieving better results. In the LLM-DQN

case, since the LLM agent had to play with another agent that was previously trained to always cooperate and had a stable behaviour, it was simpler for them to coordinate, achieving higher scores than the other case.

In these experiments all the configurations were tested for 10 episodes of 50 steps each. The mean and standard deviation of the total episode rewards and the total number of stags caught are shown in Table 1.

Model Name	Against Itself		Against DQN	
	Ep. Reward	Ep. Stags	Ep. Reward	Ep. Stags
Mistral 3 3B	18.80 (10.19)	3.70 (1.95)	47.15 (13.44)	9.10 (2.74)
Mistral 3 8B	67.00 (13.34)	13.80 (2.40)	95.50 (8.32)	18.90 (1.81)
Mistral 3 14B	74.80 (12.59)	15.20 (2.52)	95.55 (12.07)	18.90 (2.26)

Table 1: LLMs experiments results. The table shows the mean and standard deviation computed on 10 episodes of 50 steps. Values in format mean (*std*)

4 Conclusion

The results of the experiments show how RL agents can be trained via DQN to cooperate in the Stag Hunt and Escalation environments and how the cooperation becomes more difficult the more severe the punishments are. In both cases the change of objective in the maximization of the global shared reward strongly helped the agents to cooperate, learning almost always optimal behaviours. Regarding LLMs experiments, they obtained good results. As possible extensions, RL and LLM agents can be provided also with an history of previous interactions, testing them also against fixed policy agents that implement a certain strategy of cooperation/defection, or train a single RL agent to play both environments or in different grid sizes. Moreover, it would be interesting to finetune with RL an LLM agent with the rewards provided by the environment, specializing it to play better.

References

- [1] David Lvovich Nesterov-Rappoport. The evolution of trust: Understanding prosocial behavior in multi-agent reinforcement learning systems. Technical report, Drew University, Madison, NJ, May 2022.
- [2] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.
- [3] Unsloth and Mistral AI. Ministral-3-14b-reasoning-2512-gguf. <https://huggingface.co/unsloth/Ministral-3-14B-Reasoning-2512-GGUF>, 2025. Accessed via Hugging Face.
- [4] Unsloth and Mistral AI. Ministral-3-3b-reasoning-2512-gguf. <https://huggingface.co/unsloth/Ministral-3-3B-Reasoning-2512-GGUF>, 2025. Accessed via Hugging Face.
- [5] Unsloth and Mistral AI. Ministral-3-8b-reasoning-2512-gguf. <https://huggingface.co/unsloth/Ministral-3-8B-Reasoning-2512-GGUF>, 2025. Accessed via Hugging Face.
- [6] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- [7] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.